Distributed Systems

Homework 4

الاسم: هلا محمد بركات

تهدف هذه الوظيفة إلى دراسة نقل كائنات البيانات المعقدة بين عُقد الأنظمة الموزعة عبر عمليتي Serialization وDeserialization، ومقارنة ثلاث آليات: JSON وJava Standard Serialization وGoogle Protocol Buffers من حيث البنية، والتحقق من الأنواع، وحجم الرسائل وتوافقها عند تعديل المخططات.

استُخدم مشروع network-communication الأصلي بوصفه أساساً للتنفيذ، مع إضافة ملفات التجارب المطلوبة واختبارها في Code. VS ويمثل المثال ملف مشتريات عميل يتضمن اسمه والمدينة والمنتجات وأسعارها وأهليته لعروض العطلات. تُعرض في الفقرات التالية طريقة الحل، ونتائج التشغيل الموثّقة بلقطات الشاشة، ثم الإجابات عن أسئلة التقرير والمناقشة النهائية.

تتضمن الوظيفة ثلاث تجارب: (1) إرسال كائن JSON واختبار غياب Schema موحد؛ (2) تسلسل كائن Java واستعادته وفحص الحقول المستثناة والأخطاء؛ (3) تعريف ملف .proto وتوليد الأصناف واختبار الحجم وEvolution. Schema

1. التجربة الأولى: JSON ومشكلة غياب المخطط الموحد

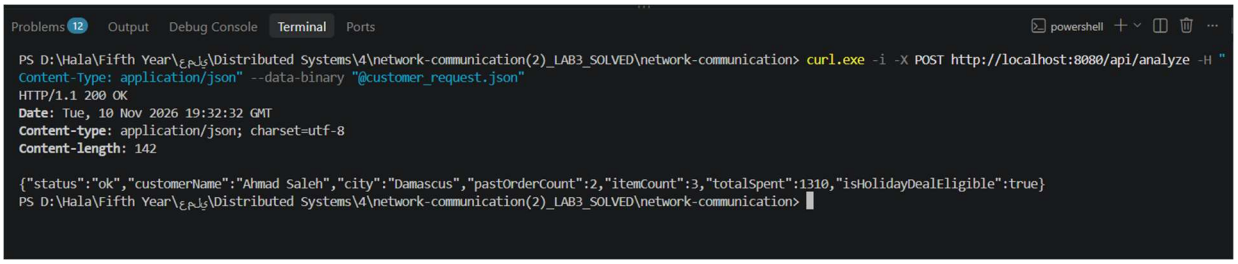
توضح هذه التجربة سهولة تمثيل البيانات المتنوعة بصيغة JSON وإرسالها عبر HTTP، ثم تكشف مشكلة قد تظهر عندما تتفق خدمتان على اسم الحقل نفسه وتختلفان في نوع البيانات المتوقّع داخله.

1.1 إنشاء كائن العميل وإرساله عبر HTTP

أُنشئ الملف customer_request.json بحيث يصف العميل Saleh Ahmad في مدينة Damascus. يحتوي على مصفوفة pastOrders من كائنين، ومصفوفة purchasedItems من ثلاثة منتجات، ومصفوفة itemPrices بالقيم [1200, 25, 85]، إلى جانب القيمة المنطقية isHolidayDealEligible = true. بذلك يمثّل الملف نصوصاً وأعداداً ومصفوفة نصوص ومصفوفة كائنات وقيمة boolean.

أُضيف المسار POST /api/analyze إلى الخادم WebServer الأصلي. عند استقبال الطلب يتحقق من نوع Content-Type ومن الحقول وأنواعها، ثم يعيد ملخصاً يتضمن اسم العميل والمدينة وعدد الطلبات السابقة وعدد المنتجات وإجمالي الأسعار.

```
curl.exe -i -X POST http://localhost:8080/api/analyze ...
```



الشكل (1): نجاح إرسال customer_request.json واستجابة HTTP **200** OK

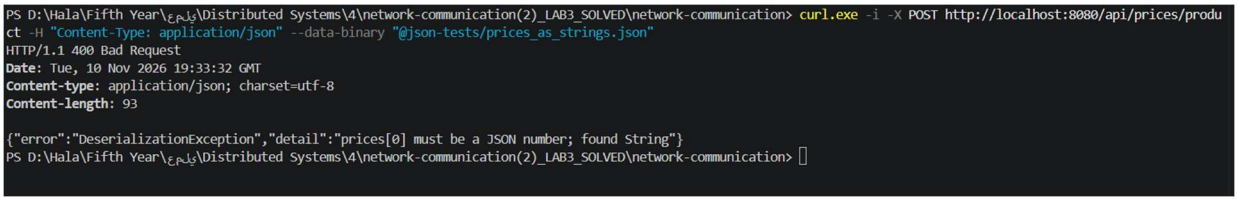
تظهر النتيجة status = ok واسم العميل Saleh Ahmad والمدينة Damascus وعدد الطلبات السابقة 2 وعدد المنتجات 3، وإجمالي الأسعار totalSpent = 1310. وتؤكد استجابة 200 OK أن الخادم استقبل البيانات وحللها بنجاح. هذا المثال يبرز سهولة فحص JSON وقراءته وتعديله وإرساله من دون أدوات خاصة بالتنسيق نفسه.

1.2 محاكاة مشكلة عدم تطابق أنواع الأسعار

لإظهار مشكلة غياب مصدر موحّد للحقيقة (Truth of Source Single)، جرى إرسال مصفوفة prices بصيغتين: الأولى أعداد مكتوبة كنصوص، بينما يتوقع المستقبل أعداداً رقمية. أما الثانية فكانت القيم الرقمية الصحيحة التي يقبلها المستقبل.

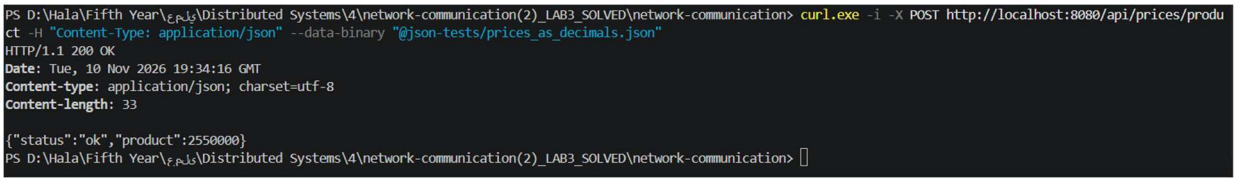
```
Sender:   {"prices": ["1200", "25", "85"]}
Receiver: {"prices": [1200.00, 25.00, 85.00]}
```

في الاختبار الأول وصل الطلب إلى api/prices/product/ لكن الخادم رفضه برسالة DeserializationException تشير إلى أن العنصر الأول من prices يجب أن يكون رقماً لا نصاً. هذا الاستثناء مُعرّف في الكود التعليمي للخادم وليس استثناءً عاماً تفرضه لغة JSON بذاتها.



الشكل (2): رفض مصفوفة الأسعار النصية: HTTP **400** ورسالة خطأ النوع

بعد تغيير القيم إلى أعداد رقمية، نجح التحليل والحساب، وأُعيد حاصل ضرب الأسعار: 1200 × 25 × 85 = 2550000.



الشكل (3): نجاح مصفوفة الأسعار الرقمية: HTTP **200** والنتيجة **2550000**

سؤال التقرير (1): لماذا لا يمكن تحديد الخدمة الصحيحة من رسالة JSON وحدها؟

لأن JSON يصف البنية والقيم المرسلة لكنه لا يفرض افتراضياً مخططاً موحداً يحدد الأنواع المتوقعة لدى جميع الخدمات. قد تعتبر خدمة الإرسال أن ["1200", "25", "85"] بيانات صحيحة، بينما تشترط خدمة الاستقبال مصفوفة أرقام. لذلك لا يمكن لمهندس الصيانة معرفة الطرف الذي خالف الاتفاق بمجرد مشاهدة الرسالة، بل يحتاج إلى عقد واجهة موثق يحدد نوع prices وطريقة التعامل مع الأخطاء.

في الأنظمة الموزعة كبيرة الحجم يؤدي غياب Schema مشترك إلى أعطال تكامل يصعب تتبعها، واختلافات بين الإصدارات، وتكرار منطق التحقق في الخدمات، وارتفاع زمن الصيانة. ويمكن معالجة المشكلة بوضع Schema JSON أو OpenAPI مع اختبارات عقود (Tests Contract) والتحقق من الأنواع قبل معالجة الطلب.

2. التجربة الثانية: Serialization Java Standard

تركز هذه التجربة على تحويل كائن Java كامل إلى تدفق بايتات ثم استعادته، مع توضيح الفرق بين الحقول العادية والحقول التي لا تدخل في عملية التسلسل، واختبار سلوك Java عند محاولة تسلسل كائن فرعي غير قابل للتسلسل.

2.1 إنشاء الصنف CustomerProfile

أُنشئ الملف CustomerProfile.java باستخدام Serializable implements وserialVersionUID = L.1 يحتوي الصنف على البيانات الأساسية customerName وpurchasedItems وitemPrices و.isHolidayDealEligible كما يحتوي على الحقل static المسمى systemVersion بقيمة v2.4، والحقل transient المسمى tempDiscountToken الذي يمثل رمز خصم مؤقتاً لا ينبغي نقله.

الحقل static يتبع الصنف ولا يمثل حالة مستقلة داخل كل كائن، ولذلك لا يُحفظ في تدفق الكائن. أما الحقل transient فيُستبعد من عملية التسلسل الافتراضية؛ وهذا يمنع تضمين الرمز المؤقت في الرسالة. أُضيف كذلك الصنف Address غير القابل للتسلسل لإجراء اختبار الخطأ في المهمة 2.3.

2.2 تنفيذ Serialization و Deserialization

استُخدم ByteArrayOutputStream مع ObjectOutputStream لتحويل CustomerProfile إلى byte[]، ثم ByteArrayInputStream مع ObjectInputStream لاسترجاع كائن مكافئ. بين العمليتين عُدّلت قيمة CustomerProfile.systemVersion من v2.4 إلى v3.0-UPDATED، حتى يمكن ملاحظة ما إذا كانت قيمة static تأتي من الرسالة أو من حالة الصنف الحالية.

```
PS D:\Hala\Fifth Year\علمع\Distributed Systems\4\network-communication(2)_LAB3_SOLVED\network-communication> cd .\java-serialization\src
PS D:\Hala\Fifth Year\علمع\Distributed Systems\4\network-communication(2)_LAB3_SOLVED\network-communication\java-serialization\src> javac *.java
PS D:\Hala\Fifth Year\علمع\Distributed Systems\4\network-communication(2)_LAB3_SOLVED\network-communication\java-serialization\src> java JavaSerializationLab
BEFORE serialization:
CustomerProfile{systemVersion='v2.4', tempDiscountToken='SECRET-TOKEN-999', customerName='Ahmad Saleh', purchasedItems=[Laptop, Mouse], itemPrices=[1200, 25], isHolida
yDealEligible=true, address=null}

Java Serialization size: 345 bytes

AFTER deserialization:
CustomerProfile{systemVersion='v3.0-UPDATED', tempDiscountToken='null', customerName='Ahmad Saleh', purchasedItems=[Laptop, Mouse], itemPrices=[1200, 25], isHolidayDea
lEligible=true, address=null}
CHECK transient token == null: true
CHECK static version == updated value: true
PS D:\Hala\Fifth Year\علمع\Distributed Systems\4\network-communication(2)_LAB3_SOLVED\network-communication\java-serialization\src>
```

الشكل (4): الكائن قبل التسلسل وبعده، وحجم رسالة Serialization Java

يُظهر التشغيل أن حجم الرسالة هو 345 bytes، وأن الاسم والمنتجات والأسعار والأهلية استُعيدت دون فقد. قبل التسلسل ظهر tempDiscountToken = SECRET-TOKEN-999 و systemVersion = v2.4. وبعد الاستعادة ظهر tempDiscountToken = null، بينما أصبحت systemVersion = v3.0-UPDATED.

2.3 الاستكشاف والأخطاء

أولاً – تفسير transient و static: تُستعاد قيمة tempDiscountToken بقيمة null لأن الحقل transient لم يُكتب في ObjectOutputStream، والقيمة الافتراضية لحقل String هي null. أما systemVersion فتبقى قيمته الجديدة v3.0-UPDATED، لأن static لا يُسلسل مع الكائن؛ وما نراه بعد القراءة هو قيمة المتغير الصنفي الموجودة حالياً في الذاكرة.

ثانياً – إضافة Address غير القابل للتسلسل: عندما وُضع كائن من الصنف Address داخل CustomerProfile وحاول البرنامج تسلسل الرسم البياني للكائنات، رُمي الاستثناء java.io.NotSerializableException: Address. يحدث ذلك لأن كل كائن غير مستثنى ضمن مسار التسلسل يجب أن يدعم Serializable أو يُعالج بآلية مخصصة.

```
PS D:\Hala\Fifth Year\يلمع\Distributed Systems\4\network-communication(2)_LAB3_SOLVED\network-communication\java-serialization\src> java JavaSerializationLab address-error
EXPECTED ERROR: java.io.NotSerializableException: Address
PS D:\Hala\Fifth Year\يلمع\Distributed Systems\4\network-communication(2)_LAB3_SOLVED\network-communication\java-serialization\src>
```

الشكل (5): ظهور NotSerializableException عند محاولة تسلسل Address

ثالثاً – طباعة Payload كنص: عند فحص String(payload) new تظهر محارف غير مقروءة مع أجزاء نصية مفهومة، مثل اسم الصنف CustomerProfile وأسماء بعض الحقول وأنواعها. كما يُثبت فحص التدفق عدم وجود النص SECRET-TOKEN-999 بفضل transient.

```
PS D:\Hala\Fifth Year\يلمع\Distributed Systems\4\network-communication(2)_LAB3_SOLVED\network-communication\java-serialization\src> java JavaSerializationLab inspect
BEFORE serialization:
CustomerProfile{systemVersion='v2.4', tempDiscountToken='SECRET-TOKEN-999', customerName='Ahmad Saleh', purchasedItems=[Laptop, Mouse], itemPrices=[1200, 25], isHolidayDealEligible=true, address=null}

Java Serialization size: 345 bytes

AFTER deserialization:
CustomerProfile{systemVersion='v3.0-UPDATED', tempDiscountToken='null', customerName='Ahmad Saleh', purchasedItems=[Laptop, Mouse], itemPrices=[1200, 25], isHolidayDealEligible=true, address=null}
CHECK transient token == null: true
CHECK static version == updated value: true

Payload as text with platform default charset (as requested in PDF):
??srCustomerProfileZisHolidayDealEligibleLaddresst      LAddress;L
                                                customerNametLjava/lang/String;[
itemPricest[ILpurchasedItemstLjava/util/List;xppt
                                        Ahmad Salehur[IM?`&v?xp?srjava.util.Arrays$ArrayList?<???[at[Ljava/lang/Object;xpur[Ljava.lang.String;??V??{GxptLaptop
tMouse

Class name found in payload: true
Field name found in payload: true
Field name found in payload: true
SECRET token found in payload: false
PS D:\Hala\Fifth Year\يلمع\Distributed Systems\4\network-communication(2)_LAB3_SOLVED\network-communication\java-serialization\src>
```

الشكل (6): فحص بايتات Serialization Java وظهور المعلومات الوصفية

تدل هذه النتيجة على أن التنسيق الثنائي ليس وسيلة لحماية المعلومات السرية؛ إذ يمكن تحليل البايتات واستخراج بيانات وصفية منها. كما أن فك تسلسل بيانات Java غير موثوقة قد يكون خطراً أمنياً، لذلك ينبغي عدم استخدام ObjectInputStream مع مدخلات خارجية غير موثوقة دون ضوابط مناسبة.

3. التجربة الثالثة: Google Protocol Buffers

تهدف التجربة الثالثة إلى استخدام مخطط Schema صريح لتحديد أسماء الحقول وأنواعها وأرقامها، ثم توليد أصناف Java منه. ويُستخدم التمثيل الثنائي في Protobuf لنقل القيم بحجم صغير دون إرسال أسماء الحقول النصية ضمن كل رسالة.

3.1 إنشاء ملف customer_profile.proto وتوليد أصناف Java

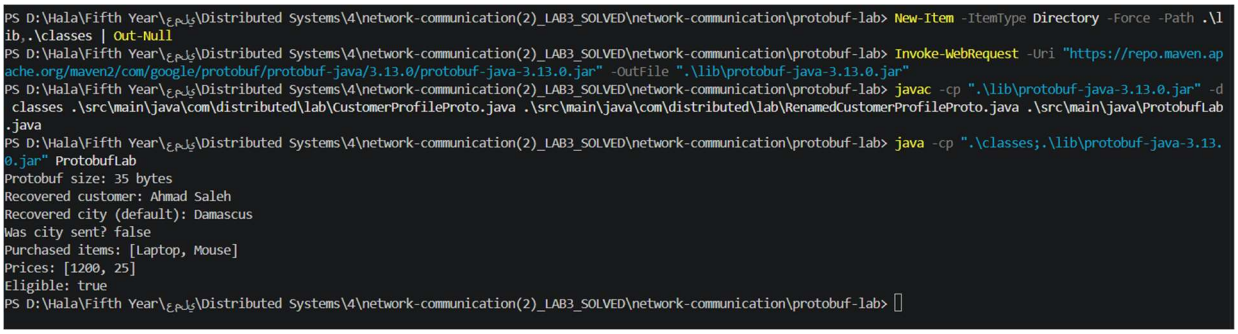
أُنشئ مخطط proto2 ضمن الحزمة distributed.lab مع تحديد java_package = com.distributed.lab وjava_outer_classname = CustomerProfileProto. يتضمن الرسالة خمسة حقول لها أرقام تعريف ثابتة:

```
required string customer_name = 1;
optional string city = 2 [default = "Damascus"];
repeated string purchased_items = 3;
repeated int32 item_prices = 4;
optional bool is_holiday_deal_eligible = 5 [default = false];
```

يُلزم required بوجود customer_name قبل بناء رسالة صحيحة في proto2، ويجعل optional المدينة والأهلية اختياريتين، ويسمح repeated بتكرار المنتجات والأسعار. ولّد protoc ملف CustomerProfileProto.java الذي يتضمن Builder والدوال getCustomerName() وgetPurchasedItemsList() وhasCity() وغيرها. تم الاحتفاظ بالأصناف المولّدة في المشروع لسهولة تشغيل الاختبارات.

3.2 بناء كائن Protobuf وقياس الحجم واستعادة القيم

استُخدم CustomerProfile.newBuilder() لوضع الاسم Saleh Ahmad والمنتجين Laptop وMouse والأسعار [1200, 25] وقيمة الأهلية true. ثم حُولت الرسالة إلى مصفوفة بايتات باستخدام toByteArray()، واستُعيدت باستخدام parseFrom(). لم تُعيّن city عمداً لاختبار القيمة الافتراضية المعرفة في Schema.

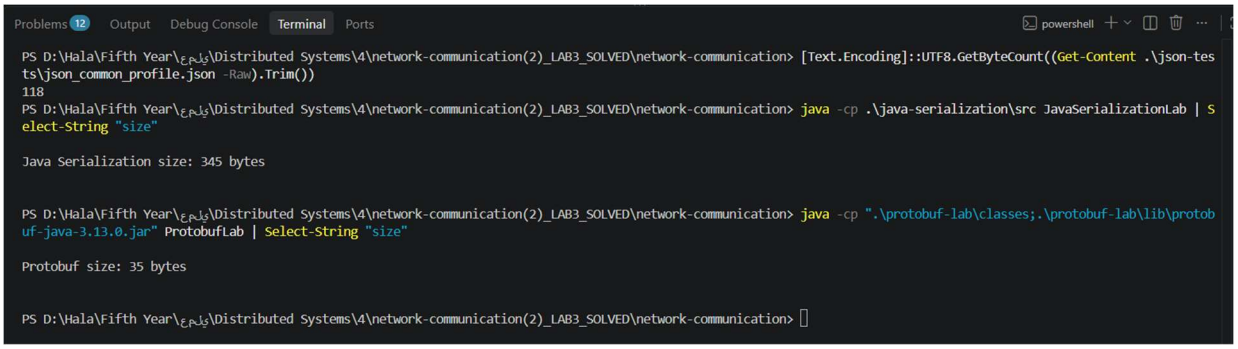


الشكل (7): تشغيل Protobuf واستخراج الحجم والبيانات المستعادة

أظهر التشغيل أن حجم الرسالة 35 bytes. أعاد getCustomerName() الاسم الصحيح، وأعادت getCity() القيمة الافتراضية Damascus، مع أن hasCity() كانت false. وهذا يثبت الفرق بين القيمة التي تُعاد بواسطة الـ getter والوجود الفعلي للحقل في الرسالة المنقولة.

سؤال التقرير (2-أ): مقارنة حجم Protobuf مع Java Serialization وJSON

لإجراء مقارنة عادلة، استُخدمت البيانات المنطقية نفسها في الحالات الثلاث: الاسم Saleh Ahmad، ومنتجان هما Laptop وMouse، والأسعار 1200 و25، والأهلية true، دون تضمين pastOrders أو المدينة الاختيارية. حُسب حجم JSON بصيغة UTF-8 دون نهاية سطر إضافية.

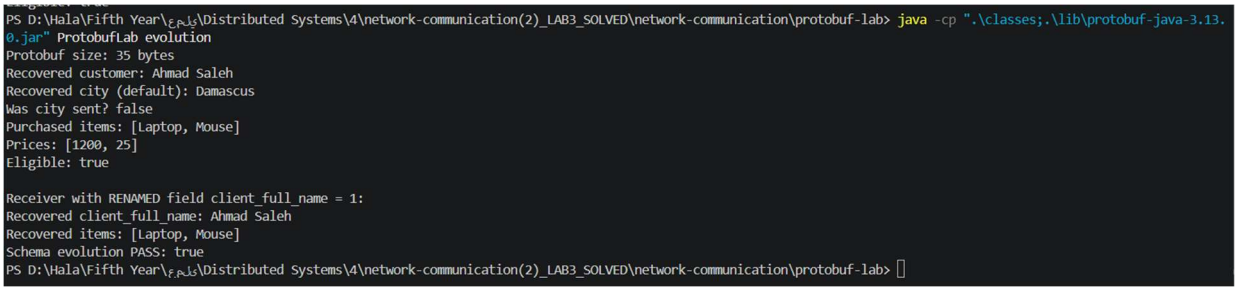


الشكل (10): مقارنة الأحجام الفعلية للرسائل المتكافئة في التنسيقات الثلاثة

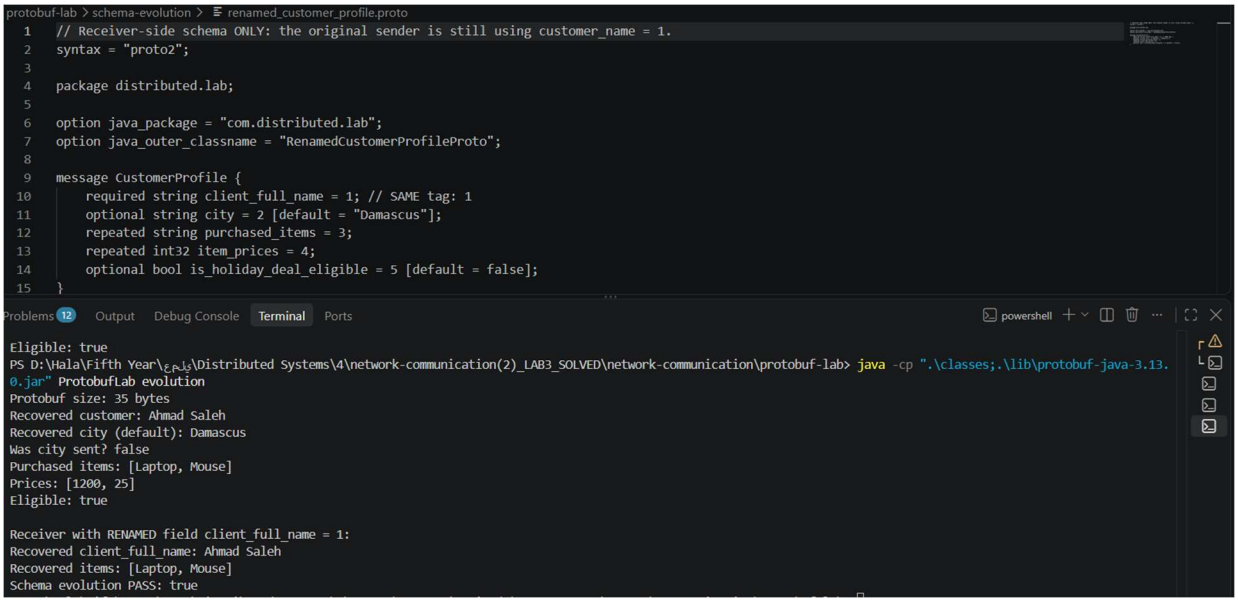
النتائج هي: JSON = 118 bytes؛ Java Serialization = 345 bytes؛ Protocol Buffers = 35 bytes. يرجع كبر JSON إلى أسماء الحقول والقيم النصية، بينما تتضمن رسالة Java بيانات وصفية عن الصنف ومكوّناته. في المقابل يستخدم Protobuf أرقام حقول (Tags Numeric) وتشفيراً ثنائياً مقتضباً للقيم. الأرقام تخص هذا المثال ولا تعني أن هذه النسب ثابتة مع كل البيانات.

3.3 اختبار Evolution Schema بتغيير اسم الحقل

نُسخ المخطط إلى ملف خاص بالمستقبل، ثم استُبدل اسم الحقل customer_name بالاسم client_full_name مع الإبقاء على رقم التعريف 1 ونوع string. لم يُعدّل مخطط المرسل. بعد إعادة توليد الصنف الخاص بالمستقبل، جرى تحليل البايتات المرسلة بالمخطط القديم باستخدام المخطط الجديد.



الشكل (8): نجاح تفسير رسالة المرسل بواسطة الصنف المعدّل في المستقبل



الشكل (9): المخطط المعدّل client_full_name = **1** ونجاح Evolution Schema

سؤال التقرير (2–ب): هل ينجح فك التسلسل بعد تغيير اسم الحقل فقط؟ ولماذا؟

نعم، نجحت العملية واستُعيد الاسم Saleh Ahmad في الحقل client_full_name. فعند تشفير Protobuf لا يُرسل الاسم النصي customer_name، بل يُرسل رقم الحقل 1 ونوع النقل والقيمة، ولذلك يستطيع المستقبل ربط الرقم 1 بالاسم الجديد ما دام النوع متوافقاً. ولا تعني هذه النتيجة جواز إعادة استخدام الأرقام لحقل مختلف المعنى، أو تغيير نوع الحقل دون دراسة قواعد التوافق.

4. أسئلة التقييم والمناقشة الختامية

بعد مقارنة آليات التسلسل، يمكن اختيار التقنية المناسبة وفق لغة الأطراف، ومستوى التوافق المطلوب، وسهولة التطوير، وحجم البيانات، والمتطلبات الأمنية. وفيما يأتي إجابات الحالات الأربع المطلوبة في ورقة الوظيفة.

4.1 الحالة الأولى: تطبيق ويب من JavaScript إلى خادم Backend

الاختيار الأنسب هو JSON. فهو مدعوم مباشرة في JavaScript عبر JSON.stringify() و JSON.parse()، ويسهل إرساله في طلبات HTTP وعرضه وتصحيحه من أدوات المطور. وهذا مناسب لتمثيل سلة مشتريات في تطبيق ويب؛ ويمكن إضافة Schema JSON للتحقق من الحقول والأنواع ومنع أخطاء التكامل. ويجب استخدام HTTPS لحماية البيانات أثناء النقل.

4.2 الحالة الثانية: نموذج أولي يتكون من ثلاث عُقد Java

الاختيار في سياق المتطلبات المعطاة هو Serialization Java Standard، لأن جميع العقد تستخدم Java وإصدار JVM نفسه، ولأن Serializable وObjectOutputStream وObjectInputStream متوفرة في JDK دون مكتبات خارجية أو مترجم Schema. يساعد ذلك على مشاركة كائنات معقدة بسرعة في نموذج أولي، لكنه يخلق ارتباطاً بتعريفات أصناف Java وإصداراتها، ولا ينبغي استخدامه لفك تسلسل بيانات غير موثوقة في نظام إنتاجي.

4.3 الحالة الثالثة: نظام تداول مالي متعدد اللغات وعالي الأداء

الاختيار الأنسب هو Buffers Protocol؛ لأنه يتيح مشاركة مخطط .proto واحد بين خدمات Java وC++ وGo، ويحدد حقول الرسالة وأنواعها وأرقامها بدقة، ويتميز بحجم نقل منخفض مقارنة بالأمثلة الأخرى. يساعد ذلك على تقليل استهلاك Bandwidth ورفع درجة التوافق بين الفرق. لكن Protobuf ليس تشفيراً ولا يوفر السرية أو التوثيق أو سلامة الاتصال بذاته، ولذلك ينبغي استخدام TLS/mTLS وضبط الهوية والصلاحيات والتحقق من المدخلات في النظام المالي.

4.4 السؤال الإضافي: لماذا لا يجوز إعادة استخدام Number Tag محذوف؟

قد تحتوي رسائل مخزنة أو برامج قديمة على رقم حقل optional حُذف في إصدار جديد. إذا استُخدم الرقم نفسه لحقل جديد ذي معنى مختلف، فقد يقرأه المستقبل على أنه الحقل الجديد ويفسر البيانات على نحو خاطئ، وقد يحدث فساد منطقي صامت يصعب اكتشافه حتى لو كان نوع النقل متوافقاً. للحفاظ على التوافق، يجب حجز الرقم المحذوف وعدم إسناده لأي حقل جديد، مع إمكانية حجز اسمه أيضاً باستخدام reserved في ملف ..proto

```
reserved 6;
reserved "removed_field";
```

5. الخاتمة

أثبتت التجارب أن JSON سهل القراءة والنقل لكنه يحتاج إلى عقد متفق عليه لمنع تضارب الأنواع؛ وأن Serialization Java يتيح نقل كائنات Java كاملة مع استبعاد حقول transient وstatic لكنه يرتبط ببنية الأصناف ويكشف معلومات وصفية؛ وأن Buffers Protocol يوفر مخططاً واضحاً وتمثيلاً صغير الحجم وتوافقاً أفضل عند تغييرات معينة في أسماء الحقول. وفي بيانات الاختبار المشتركة كان Protobuf أصغر رسالة بحجم 35 bytes، مقارنة بـ118 bytes لـJSON و345 bytes لـJava .Serialization

تعتمد التقنية الأنسب على طبيعة النظام: JSON لتطبيقات الويب، وSerialization Java لنموذج Java داخلي سريع بشروطه التعليمية، وBuffers Protocol للخدمات المتعددة اللغات وعالية الأداء. في جميع الأحوال يجب التعامل مع التحقق من المخططات وأمن النقل بوصفهما متطلبين مستقلين عن مجرد اختيار صيغة التسلسل.